

Specyfikacja techniczna

Aplikacja **ContactManager**

Wielowarstwowe API REST (ASP.NET Core 10)
z frontendem React 19 + TypeScript

Autor: Filip Pudlak

Data: 25 czerwca 2026

Wersja: 1.0

Spis treści

1	Wprowadzenie i architektura	3
2	Opis klas i metod	3
2.1	Warstwa Domain	3
2.2	Warstwa Application	5
2.3	Warstwa Infrastructure	7
2.4	Warstwa Api	8
3	Wykorzystane biblioteki	10
3.1	Backend (.NET 10)	10
3.2	Testy	10
3.3	Frontend (Node / Vite)	10
4	Sposób kompilacji i uruchomienia	10
4.1	Wymagania	10
4.2	Krok po kroku	11
4.3	Testy	11
4.4	Formatowanie i CI	11
4.5	Konfiguracja i alternatywa	11
5	Frontend (skrót)	11

1 Wprowadzenie i architektura

ContactManager to wielowarstwowa aplikacja webowa służąca do zarządzania kontaktami. Część serwerowa to API REST zbudowane w **ASP.NET Core 10.0**, korzystające z bazy danych **PostgreSQL**, z uwierzytelnianiem opartym o tokeny **JWT** oraz mechanizm odświeżania (refresh token) z rotacją i wykrywaniem kradzieży tokenu. Część kliencka to aplikacja jednostronicowa (SPA) napisana w **React 19 + TypeScript** (bundler **Vite**).

Backend zaprojektowano zgodnie z zasadami **Clean Architecture** — kod podzielono na cztery warstwy o zależnościach skierowanych wyłącznie „do wewnątrz”:

Warstwa	Odpowiedzialność
Domain	Encje, reguły biznesowe, wyjątki dziedzinowe, interfejsy repozytoriów. Brak zależności zewnętrznych.
Application	Logika aplikacyjna: usługi, DTO, walidatory, interfejsy usług. Zależy tylko od Domain.
Infrastructure	Implementacje techniczne: dostęp do bazy (EF Core), hashowanie haseł, generowanie tokenów. Zależy od Domain i Application.
Api	Warstwa prezentacji HTTP: kontrolery, obsługa wyjątków, konfiguracja (DI, JWT, CORS, OpenAPI), start aplikacji.

Kierunek zależności między projektami:

Api → Application → Domain
Api → Infrastructure → (Application, Domain)

Zastosowane wzorce projektowe: **Clean Architecture**, **Repository**, **Service Layer**, **DTO**, **Dependency Injection**, walidacja (**FluentValidation**), uwierzytelnianie **JWT** z rotacją refresh tokenów, **optimistic concurrency** oraz globalna obsługa wyjątków mapowana na `ProblemDetails`.

2 Opis klas i metod

2.1 Warstwa Domain

Lokalizacja: `src/ContactManager.Domain/`.

2.1.1 Encje

User — `Entities/User.cs`

Konto uwierzytelnionego użytkownika systemu (dane logowania i metadane).

Właściwość	Typ	Opis
Id	Guid	Identyfikator użytkownika.
Email	string	Adres e-mail (login), wymagany.
FirstName / LastName	string?	Imię i nazwisko (opcjonalne).
PasswordHash	string	Hasło zahashowane algorytmem BCrypt.
CreatedAt	DateTime	Data utworzenia konta (UTC).
RowVersion	uint	Znacznik współbieżności (xmin).

Contact — `Entities/Contact.cs`

Rekord kontaktu z pełnym profilem, klasyfikacją kategoria/podkategoria oraz hasłem. Kontakt jest powiązany (po e-mailu) z kontem logowania `User`: utworzenie kontaktu zakłada konto o tym samym e-mailu i hasle, zmiana e-maila/hasła aktualizuje konto, a usunięcie kontaktu je kasuje — dzięki czemu kontakt może zalogować się własnymi danymi.

Właściwość	Typ	Opis
Id	Guid	Identyfikator kontaktu.
FirstName / LastName	string	Imię i nazwisko (wymagane).
Email	string	Adres e-mail, unikalny.
PasswordHash	string	Hasło kontaktu (BCrypt).
Phone	string	Numer telefonu.
BirthDate	DateOnly	Data urodzenia.
CategoryId / Category	int / Category	Kategoria (wymagana) + nawigacja.
SubcategoryId / Subcategory	int? / Subcategory?	Podkategoria słownikowa (opcjonalna).
CustomSubcategory	string?	Podkategoria tekstowa (gdy kategoria na to pozwala).
CreatedAt / UpdatedAt	DateTime / DateTime?	Daty utworzenia i modyfikacji (UTC).
RowVersion	uint	Znacznik współbieżności (xmin).

Category — Entities/Category.cs

Encja słownikowa kategorii kontaktu (np. *Służbowy*, *Prywatny*, *Inny*). Pole `AllowsCustomSubcategory` decyduje, czy dla kategorii dozwolona jest podkategoria tekstowa. Właściwości: `Id`, `Name` (unikalna), `AllowsCustomSubcategory`, kolekcje `Subcategories` i `Contacts`.

Subcategory — Entities/Subcategory.cs

Encja słownikowa podkategorii w obrębie kategorii (np. *Szef*, *Klient*, *Pracownik*, *Kontrahent* dla kategorii *Służbowy*). Właściwości: `Id`, `Name`, `CategoryId`, nawigacja `Category`, kolekcja `Contacts`. Unikalność pary (`CategoryId`, `Name`).

RefreshToken — Entities/RefreshToken.cs

Sesja tokenu odświeżającego. Przechowywany jest wyłącznie hash tokenu; wartość jawna trafia jednorazowo do klienta (cookie `HttpOnly`). Umożliwia rotację tokenów i wykrycie kradzieży.

Właściwość	Typ	Opis
Id	Guid	Identyfikator tokenu.
UserId / User	Guid / User?	Powiązany użytkownik.
TokenHash	string	Hash SHA-256 wartości tokenu (nigdy jawny).
ExpiresAtUtc / CreatedAtUtc	DateTime	Czas wygaśnięcia i utworzenia (UTC).
RevokedAtUtc	DateTime?	Czas unieważnienia (null = aktywny).
ReplacedByTokenHash	string?	Hash tokenu-następcy (śledzenie łańcucha rotacji).
IsActive	bool	Właściwość wyliczana: token nieunieważniony i niewygasły.

2.1.2 Wyjątki dziedziczne

Lokalizacja: Exceptions/. Każdy mapowany jest przez GlobalExceptionHandler na odpowiedni kod HTTP.

Klasa	HTTP	Znaczenie
BusinessRuleViolationException	400	Naruszenie reguły biznesowej (np. błędna para kategoria/podkategoria).
ForbiddenActionException	403	Brak uprawnień do operacji (np. zmiana hasła cudzego konta).
EmailConflictException	409	Kontakt o podanym e-mailu już istnieje.
ConcurrencyConflictException	409	Konflikt współbieżności — rekord zmieniony między odczytem a zapisem.

2.1.3 Interfejsy repozytoriów

Lokalizacja: Repositories/. Definiują kontrakty dostępu do danych (implementacje w warstwie Infrastructure).

IUserRepository

- Task<User?> GetByEmailAsync(string email, CancellationToken) — zwraca użytkownika po e-mailu lub null.

IContactRepository

- Task<IReadOnlyList<Contact>> GetAllAsync(CancellationToken) — wszystkie kontakty (z dołączonymi słownikami).
- Task<Contact?> GetByIdAsync(Guid id, CancellationToken) — kontakt po identyfikatorze (śledzony) lub null.
- Task<Contact?> GetByEmailAsync(string email, CancellationToken) — kontakt po e-mailu (kontrola unikalności).
- Task AddAsync(Contact, User loginUser, CancellationToken) — dodanie kontaktu wraz z kontem logowania (jeden zapis/transakcja).
- Task UpdateAsync(Contact, uint expectedRowVersion, CancellationToken) — aktualizacja z kontrolą współbieżności; rzuca ConcurrencyConflictException przy niezgodności RowVersion.
- Task DeleteAsync(Contact, User? loginUser, CancellationToken) — usunięcie kontaktu wraz z kontem logowania (jeden zapis/transakcja).

ICategoryRepository

- Task<IReadOnlyList<Category>> GetAllWithSubcategoriesAsync(CancellationToken) — kategorie wraz z podkategoriami.
- Task<Category?> GetByIdWithSubcategoriesAsync(int id, CancellationToken) — kategoria z podkategoriami lub null.

IRefreshTokenRepository

- Task AddAsync(RefreshToken, CancellationToken) — dodanie tokenu do kontekstu.
- Task<RefreshToken?> GetByTokenHashAsync(string tokenHash, CancellationToken) — token po hashu (z użytkownikiem) lub null.
- Task RevokeAllActiveForUserAsync(Guid userId, DateTime revokedAtUtc, CancellationToken) — unieważnienie wszystkich aktywnych tokenów użytkownika (reakcja na kradzież).
- Task SaveChangesAsync(CancellationToken) — utrwalenie zmian.

2.2 Warstwa Application

Lokalizacja: src/ContactManager.Application/.

2.2.1 Interfejsy usług

IAuthService — uwierzytelnianie.

- `Task<AuthResult?> LoginAsync(LoginRequestDto, CancellationToken)` — logowanie; para tokenów lub null przy błędnych danych.
- `Task<AuthResult?> RefreshAsync(string? rawRefreshToken, CancellationToken)` — wymiana refresh tokenu na nową parę (rotacja); null gdy token nieznany/wygasł/unieważniony; ponowne użycie zrotowanego tokenu traktowane jako kradzież.
- `Task LogoutAsync(string? rawRefreshToken, CancellationToken)` — unieważnienie tokenu (operacja idempotentna).

IContactService — logika kontaktów.

- `Task<IReadOnlyList<ContactResponseDto>> GetAllAsync(CancellationToken)` — lista wszystkich kontaktów.
- `Task<ContactResponseDto?> GetByIdAsync(Guid id, CancellationToken)` — kontakt po id lub null.
- `Task<ContactResponseDto> CreateAsync(CreateContactRequestDto, CancellationToken)` — utworzenie kontaktu (hashowanie hasła) wraz z kontem logowania; `EmailConflictException` przy e-mailu zajęтым w kontaktach lub kontakcie.
- `Task<ContactResponseDto> UpdateAsync(Guid id, UpdateContactRequestDto, CancellationToken)` — aktualizacja (zmiana e-maila synchronizuje konto logowania); null gdy brak; `EmailConflictException` / `ConcurrencyConflictException`.
- `Task<bool> DeleteAsync(Guid id, CancellationToken)` — usunięcie kontaktu wraz z kontem logowania; false gdy brak.
- `Task<bool> ChangePasswordAsync(Guid id, ChangeContactPasswordRequestDto, string callerEmail, CancellationToken)` — zmiana hasła kontaktu i konta logowania; dozwolona tylko właścicielowi (`callerEmail` = e-mail kontaktu), inaczej `ForbiddenAccessException`; false gdy brak; kontrola `RowVersion`.

ICategoryService

- `Task<IReadOnlyList<CategoryResponseDto>> GetAllAsync(CancellationToken)` — kategorie z podkategoriami.

IPasswordHasher

- `string Hash(string password)` — hash hasła.
- `bool Verify(string password, string passwordHash)` — weryfikacja hasła względem hashu.

ITokenService

- `AccessToken CreateAccessToken(User user)` — podpisany JWT (HMAC-SHA256).
- `RefreshTokenInfo GenerateRefreshToken()` — kryptograficznie losowy refresh token.
- `string HashRefreshToken(string rawToken)` — hash SHA-256 wartości tokenu.

2.2.2 Implementacje usług

ContactService — `Services/ContactService.cs`

Orkiestruje repozytorium kontaktów, waliduje reguły kategoria/podkategoria, hashuje hasła, utrzymuje spójność z kontem logowania (`User`) i mapuje encje na DTO. Konstruktor: (`IContactRepository`, `ICategoryRepository`, `IUserRepository`, `IPasswordHasher`). Metody prywatne: `ValidateCategorySelectionAsync(...)` — sprawdza poprawność pary kategoria/podkategoria względem reguł w bazie (rzuci `BusinessRuleViolationException`, zwraca znormalizowaną podkategorię tekstową); `ToResponse(Contact)` — mapowanie na DTO (bez hashu hasła).

AuthService — `Services/AuthService.cs`

Weryfikuje dane logowania, wydaje pary tokenów, realizuje rotację refresh tokenów z wykrywaniem kradzieży. Konstruktor: (`IUserRepository`, `IRefreshTokenRepository`, `IPasswordHasher`, `ITokenService`). Metody prywatne: `IssueTokensAsync(User, ...)` — tworzy access token, generuje refresh token, zapisuje jego hash i zwraca parę; `CreateTokenEntity(...)` — fabryka encji `RefreshToken`.

CategoryService — Services/CategoryService.cs

Odczytuje kategorie z repozytorium i mapuje na DTO (ToResponse(Category) z uporządkowanymi podkategoriami). Konstruktor: (ICategoryRepository).

2.2.3 Modele (obiekty wartości)

Rekord	Zawartość
AccessToken	Token, ExpiresAtUtc — wygenerowany JWT i czas wygaśnięcia.
AuthResult	AccessToken, AccessTokenExpiresAtUtc, Email, RefreshToken, RefreshTokenExpiresAtUtc — kompletny wynik uwierzytelnienia.
RefreshTokenInfo	RawToken, TokenHash, ExpiresAtUtc — wygenerowany refresh token (jawny + hash).

2.2.4 Obiekty transferu danych (DTO)

DTO	Przeznaczenie i pola
LoginRequestDto	Wejście logowania: Email, Password.
LoginResponseDto	Wyjście logowania: Token, ExpiresAtUtc, Email.
CreateContactRequestDto	Tworzenie kontaktu: dane profilu + Password, CategoryId, SubcategoryId?, CustomSubcategory?.
UpdateContactRequestDto	Aktualizacja kontaktu (bez hasła) + RowVersion.
ChangeContactPasswordRequestDto	Zmiana hasła: NewPassword, RowVersion.
ContactResponseDto	Reprezentacja kontaktu (bez hashu): dane profilu, nazwy kategorii/podkategorii, daty, RowVersion.
CategoryResponseDto	Kategoria: Id, Name, AllowsCustomSubcategory, lista Subcategories.
SubcategoryResponseDto	Podkategoria: Id, Name.

2.2.5 Walidatory (FluentValidation)

Walidator	Reguły
CreateContactRequestValidator	Imię/nazwisko (1–100), e-mail (format, 1–256), hasło (polityka złożoności), telefon (1–32), data urodzenia (przeszłość), kategoria (wymagana), podkategoria tekstowa (0–100).
UpdateContactRequestValidator	Jak wyżej, bez hasła.
ChangeContactPasswordRequestValidator	Polityka złożoności nowego hasła.
LoginRequestValidator	E-mail (wymagany, poprawny format) i hasło (niepuste).
PasswordRules	Wspólna polityka hasła: min. 8 znaków, ≥1 wielka litera, ≥1 znak specjalny. Metoda rozszerzająca ValidPassword<T>().

Rejestrację usług i walidatorów wykonuje `DependencyInjection.AddApplication(...)` (usługi Scoped, walidatory wykrywane automatycznie ze złożeń).

2.3 Warstwa Infrastructure

Lokalizacja: src/ContactManager.Infrastructure/.

2.3.1 Uwierzytelnianie

BcryptPasswordHasher (IPasswordHasher) — Auth/BcryptPasswordHasher.cs

Hashowanie i weryfikacja haseł przy użyciu biblioteki BCrypt.NET (Hash, Verify).

JwtTokenService (ITokenService) — Auth/JwtTokenService.cs

Generuje podpisane tokeny JWT (HMAC-SHA256) oraz kryptograficznie losowe refresh tokeny. Konstruktor: (IOptions<JwtSettings>, IOptions<RefreshTokenSettings>).

- `CreateAccessToken(User)` — JWT z oświadczeniami (Sub, Email, GivenName, FamilyName, Jti), podpis HMAC-SHA256, czas życia z `JwtSettings`.
- `GenerateRefreshToken()` — 32 losowe bajty kodowane Base64 URL-safe, hash SHA-256, czas życia z `RefreshTokenSettings`.
- `HashRefreshToken(string)` — szesnastkowy hash SHA-256.

JwtSettings — `Auth/JwtSettings.cs` (sekcja `Jwt`): `Issuer`, `Audience`, `Key` (min. 32 znaki dla HMAC-SHA256), `ExpiryMinutes` (domyślnie 60).

RefreshTokenSettings — `Auth/RefreshTokenSettings.cs` (sekcja `RefreshToken`): `ExpiryDays` (7), `CookieName`, `CookieSecure`, `CookieSameSite`, `CookiePath`.

2.3.2 Dostęp do danych (EF Core)

AppDbContext (`DbContext`) — `Persistence/AppDbContext.cs`

Kontekst Entity Framework Core dla PostgreSQL. Zbiory: `Users`, `RefreshTokens`, `Contacts`, `Categories`, `Subcategories`. Metoda `OnModelCreating` stosuje wszystkie konfiguracje `EntityTypeConfiguration` ze złożeńia.

DesignTimeDbContextFactory — `Persistence/DesignTimeDbContextFactory.cs`

Fabryka kontekstu dla narzędzi (migracje EF Core); czyta connection string ze środowiska lub używa wartości domyślnej dewelopera.

DataSeeder — `Persistence/Seed/DataSeeder.cs`

Inicjalizacja bazy: `SeedAdminUserAsync(...)` tworzy konto administratora (dane z sekcji `Admin`), `SeedContactsAsync(...)` dodaje przykładowe kontakty wraz z kontami logowania.

Konfiguracje encji (`Persistence/Configurations/`, Fluent API): `UserConfiguration`, `ContactConfiguration` (relacje do `Category/Subcategory` z regułą `Restrict`, unikalność e-mail), `CategoryConfiguration` (seed: *Służbowy*, *Prywatny*, *Inny*), `SubcategoryConfiguration` (seed podkategorii *Służbowy*; unikalność (`CategoryId`, `Name`)), `RefreshTokenConfiguration` (unikalny indeks `TokenHash`, kaskada do `User`). Wszystkie konfiguruja też znacznik współbieżności (`xmin`).

2.3.3 Repozytoria

Implementacje interfejsów z warstwy `Domain` (konstruktor każdej przyjmuje `AppDbContext`):

Klasa	Charakterystyka
<code>UserRepository</code>	Wyszukiwanie użytkownika po e-mailu.
<code>ContactRepository</code>	CRUD kontaktów z dołączaniem (<code>Include</code>) <code>Category/Subcategory</code> ; odczyty <code>AsNoTracking</code> ; <code>UpdateAsync</code> ustawia oryginalną wartość <code>RowVersion</code> i mapuje <code>DbUpdateConcurrencyException</code> na <code>ConcurrencyConflictException</code> .
<code>CategoryRepository</code>	Odczyt kategorii z podkategoriami (<code>AsNoTracking</code> , sortowanie po <code>Id</code>).
<code>RefreshTokenRepository</code>	Cykl życia tokenów: dodanie, wyszukiwanie po hashu, masowe unieważnianie, zapis.

Rejestrację warstwy wykonuje `DependencyInjection.AddInfrastructure(services, configuration): AppDbContext` (`PostgreSQL/Npgsql`), `BcryptPasswordHasher` i `JwtTokenService` (`Singleton`), repozytoria (`Scoped`).

2.4 Warstwa Api

Lokalizacja: `src/ContactManager.Api/`.

2.4.1 Kontrolery

AuthController – trasa `api/auth`. Access token w treści odpowiedzi; refresh token w cookie `HttpOnly` (flagi `Secure`, `SameSite`, `Path`).

Metoda	HTTP	Działanie
Login	POST	Walidacja → <code>LoginAsync</code> → 200 + <code>LoginResponseDto</code> + cookie, lub 401. Ograniczenie liczby prób (rate limiting) → 429 + nagłówek <code>Retry-After</code> .
Refresh	POST	Odczyt cookie → <code>RefreshAsync</code> → 200 + nowa para + odświeżone cookie, lub 401.
Logout	POST	Unieważnienie tokenu, usunięcie cookie → 204 (idempotentne).

Metody pomocnicze: `IssueTokens`, `SetRefreshTokenCookie`, `DeleteRefreshTokenCookie`, `BuildCookieOptions`.

ContactsController – trasa `api/contacts`. Odczyty publiczne, modyfikacje autoryzowane (JWT).

Metoda	HTTP	Dostęp	Działanie
<code>GetAll</code>	GET	Anon.	200 + lista <code>ContactResponseDto</code> .
<code>GetById</code>	GET {id}	Anon.	200 + <code>ContactResponseDto</code> lub 404.
Create	POST	Auth	Walidacja → <code>CreateAsync</code> → 201 (nagłówek <code>Location</code>) lub 400/409.
Update	PUT {id}	Auth	Walidacja → <code>UpdateAsync</code> → 204 lub 404/409/400.
ChangePassword	PUT {id}/password	Auth (właściciel)	Walidacja → <code>ChangePasswordAsync</code> → 204 lub 403 (cudze konto)/404/409/400. E-mail wywołującego z claimu JWT.
Delete	DELETE {id}	Auth	<code>DeleteAsync</code> → 204 lub 404.

CategoriesController – trasa `api/categories`. Publiczny odczyt słownika.

- `GetAll` (GET, anonimowy) → 200 + lista `CategoryResponseDto` (z `AllowsCustomSubcategory` i podkategoriami).

2.4.2 Obsługa wyjątków i OpenAPI

GlobalExceptionHandler (`ExceptionHandler`) – `Errors/GlobalExceptionHandler.cs`

Middleware przechytujące nieobsłużone wyjątki i mapujące je na odpowiedzi `ProblemDetails`. Wyjątki dziedzinowe → konkretne kody (400/403/409); pozostałe → 500 bez ujawniania szczegółów (z logowaniem). Metoda `TryHandleAsync(HttpContext, Exception, CancellationToken)`.

BearerSecuritySchemeTransformer (`IOpenApiDocumentTransformer`) – `OpenApi/...`

Rejestruje schemat zabezpieczeń JWT Bearer w dokumencie OpenAPI (przycisk „Authorize” w UI dokumentacji). Metoda `TransformAsync(...)`.

2.4.3 Start aplikacji

Program.cs – punkt wejścia hosta `WebApplication`. Rejestruje: kontrolery, OpenAPI (z `BearerSecuritySchemeTransformer`), health checks, CORS (`FrontendCorsPolicy`), `ProblemDetails` + `GlobalExceptionHandler`, usługi warstw (`AddApplication`, `AddInfrastructure`), uwierzytelnianie JWT Bearer (walidacja issuer/audience/lifetime/key, HMAC-SHA256), autoryzację oraz rate limiting endpointu logowania (okno stałe, partycjonowane po IP klienta; sekcja konfiguracji `RateLimiting:Auth`, odpowiedź 429 z nagłówkiem `Retry-After`). Potok middleware: obsługa wyjątków → OpenAPI (dev) → CORS → rate limiter → uwierzytelnianie → autoryzacja → kontrolery → health checks. Przy starcie wykonuje (opcjonalnie) automatyczne migracje i seed danych. Klasa `Program` jest publiczna na potrzeby testów integracyjnych (`WebApplicationFactory<Program>`).

3 Wykorzystane biblioteki

3.1 Backend (.NET 10)

Projekt / Pakiet	Wersja	Zastosowanie
ContactManager.Api		
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.9	Walidacja tokenów JWT (bearer).
Microsoft.AspNetCore.OpenApi	10.0.9	Generowanie schematu OpenAPI.
Microsoft.EntityFrameworkCore.Design	10.0.4	Narzędzia i migracje EF Core.
Scalar.AspNetCore	2.16.5	Interaktywne UI dokumentacji API.
ContactManager.Application		
FluentValidation.DependencyInjectionExtensions	12.1.1	Walidacja danych wejściowych + integracja z DI.
ContactManager.Infrastructure		
BCrypt.Net-Next	4.2.0	Hashowanie haseł.
Npgsql.EntityFrameworkCore.PostgreSQL	10.0.2	Dostawca bazy PostgreSQL dla EF Core.
System.IdentityModel.Tokens.Jwt	8.19.1	Tworzenie i obsługa tokenów JWT.
Microsoft.EntityFrameworkCore.Design	10.0.4	Narzędzia i migracje EF Core.
ContactManager.Domain		
—	—	Brak zależności zewnętrznych.

3.2 Testy

Pakiet	Wersja	Zastosowanie
xunit	2.9.3	Framework testowy.
xunit.runner.visualstudio	3.1.4	Uruchamianie testów w IDE.
Microsoft.NET.Test.Sdk	17.14.1	Infrastruktura SDK testów.
NSubstitute	5.3.0	Tworzenie atrap (mock) — testy jednostkowe.
Microsoft.AspNetCore.Mvc.Testing	10.0.9	WebApplicationFactory — testy integracyjne.
Testcontainers.PostgreSql	4.12.0	Izolowana baza PostgreSQL w kontenerze.
coverlet.collector	6.0.4	Pomiar pokrycia kodu.

3.3 Frontend (Node / Vite)

React 19 (react, react-dom), react-router-dom 7 (routing), axios (klient HTTP do API). Narzędzia: **Vite** 8 (bundler/serwer dev), **TypeScript** 6, **ESLint** 10 (+ wtyczki React), **Prettier** (formatowanie).

4 Sposób kompilacji i uruchomienia

Projekt korzysta z dwóch task runnerów: **vpr** (= vp run, Vite+) uruchamia skrypty z package.json, a **mise** — zadania infrastrukturalne z mise.toml. Zalecane środowisko to **dev container** (.devcontainer/Dockerfile), którego obraz zawiera .NET 10 SDK, Node 22, Vite+ oraz mise.

4.1 Wymagania

- .NET SDK **10.0.301** (przypięte w global.json, rollForward: latestFeature).
- PostgreSQL (uruchamiany w kontenerze przez mise).
- Node.js 22 + Vite+ (frontend).
- Docker (baza danych / opcjonalnie pełny stack).

Ustawienia kompilatora (Directory.Build.props): `LangVersion = latest (C# 14+)`, `Nullable = enable`, `ImplicitUsings = enable`.

4.2 Krok po kroku

1. **Sklonuj repozytorium** i otwórz je w kontenerze (VS Code „Dev Containers”). Po utworzeniu kontenera automatycznie wykonuje się `vpr install` (`postCreateCommand`).
2. **Instalacja zależności** — `vpr install`
(`dotnet restore && dotnet tool restore && (cd frontend && vp install)`).
3. **Baza danych** — `mise db:up`
(`docker compose -f docker/docker-compose.infra.yml up -d postgres`).
Pomocniczo: `mise db:down`, `mise db:reset`, `mise db:logs`.
Migracje EF Core: `mise ef:add`, `mise ef:update`.
4. **Uruchomienie frontendu i backendu** — `vpr dev`
(równoległe: `backend dotnet watch run` z `hot-reload` + `frontend vp dev/Vite`).
5. **Build produkcyjny** — `vpr build`
(`dotnet build ContactManager.slnx -c Release && (cd frontend && vp build)`; artefakty frontendu w `frontend/dist/`).

4.3 Testy

Skrypty testowe wywołują `dotnet test`:

Polecenie	Zakres
<code>vpr test</code>	Wszystkie testy (<code>dotnet test ContactManager.slnx</code>).
<code>vpr test:unit</code>	Testy jednostkowe (<code>ContactManager.UnitTests</code> , <code>xUnit</code> + <code>NSubstitute</code>).
<code>vpr test:integration</code>	Testy integracyjne (<code>ContactManager.IntegrationTests</code> , <code>WebApplicationFactory</code> + <code>Testcontainers PostgreSQL</code>).

4.4 Formatowanie i CI

Formatowanie kodu: `vpr format` (backend + frontend), `vpr format:backend`, `vpr format:frontend`.

Ciągła integracja (GitHub Actions, `.github/workflows/`):

- `ci.yml` — walidacja każdego PR do main: build i testy backendu (z `Testcontainers PostgreSQL`) oraz lint, typecheck i build frontendu.
- `release.yml` — na tagu `v*` buduje i publikuje obrazy Docker (API i frontend) do GHCR oraz tworzy wydanie GitHub.

4.5 Konfiguracja i alternatywa

Zmienne środowiskowe definiuje się w pliku `.env` (na bazie `.env.example`): połączenie do PostgreSQL, ustawienia JWT i refresh tokenu.

Alternatywnie cały stos (API + frontend + baza) można uruchomić w kontenerach: `docker compose -f docker/docker-compose.dev.yml up --build` (skrót: `vpr docker:up`).

5 Frontend (skrót)

Aplikacja kliencka (`frontend/`) to SPA w **React 19** + **TypeScript**, budowana narzędziem **Vite**. Komunikacja z API odbywa się przez **axios** (serwer dev proxuje ścieżkę `/api` na backend `http://localhost:5298`), a nawigację obsługuje **react-router-dom**. Polecenia: `vp dev` (serwer deweloperski), `vp build` (build produkcyjny do `frontend/dist/`), `vp lint` (ESLint), `vp format` (Prettier). W produkcji statyczny build serwuje nginx (`frontend/nginx.conf`) z nagłówkami bezpieczeństwa (CSP, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`) oraz proxy `/api` do backendu. Token dostępu jest przechowywany w pamięci (odporność na XSS), a sesję wznawia cookie `HttpOnly` z refresh tokenem. Opis poszczególnych komponentów wykracza poza zakres niniejszej specyfikacji.